



Computer Games Development CW208

Technical Design Document

Year IV

[Jake Fitzgerald]
[C00288105]

[Student Name]
[Student Number]

[13/04/2026]

[Declaration form to be attached]

Contents

Technical Design.....	2
Technology Stack.....	2
UML Diagram.....	3
Midi Parsing.....	4
Midi Writing.....	4
Midi Input.....	5
References.....	6

Midi

- Midi Architecture
- Midi Parser
- Midi Input
- Midi Writer

Sound

- Sound Manager
- Collision Manager

Technical Design

Technology Stack

Architecture: Windows

Language: C++

External Libraries: SFML3.0 (visuals, sound), Windows Multimedia Library (Midi API)

Server/Database: Microsoft Azure SQL Server/ SQL Database

Every game object is created inside the Game.h's constructor so that we are able to pass the different object instances to the different scenes as pointers when the game is first ran.

Object list:

- Sound Manager
- Collision Manager
- Score Manager
- Midi Writer
- Midi Parser
- Database

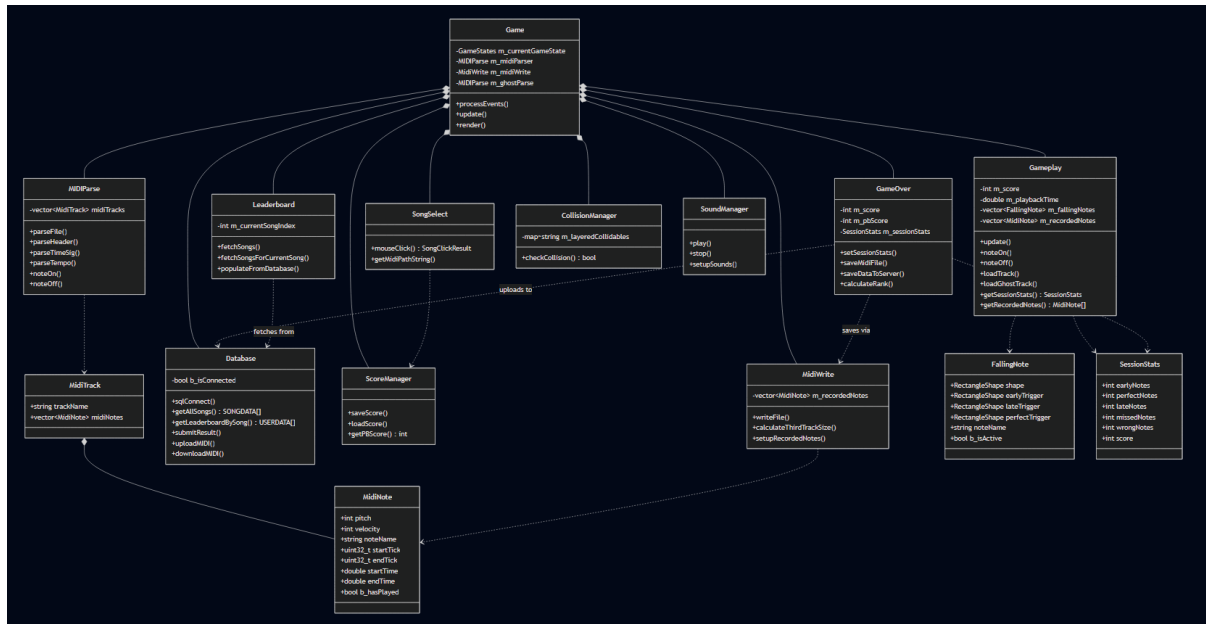
Scene State Machine enum list:

- Main Menu
- Options Menu
- Song Selection
- Visualiser Selection
- Midi Selection
- Leaderboards
- Results

The application is built for windows and needs to use the winmm.lib (Windows Multimedia Library) inside of the pc's system32 to be able to handle the Midi input from a Midi device.

It records the notes that were played during the gameplay scene which can then be saved to a Midi file either locally or to a server's database.

UML Diagram:



Midi Parsing

Read the file through `std::ifstream` as binary so that we can go through each byte and correctly store that data that we need (Track data (3 tracks), Tempo, BPM, note variables).

Functions:

`Parse()` - Opens the file for parsing and read it in binary mode. Calls the other functions for reading in sequence.

`ParseHeader()` - Reads the first few bytes to see if it's a string with "Mthd" (Midi Header) to confirm it is a valid midi file. Checks the Midi version for determining how many tracks there can be.

`Time Signature()` - Reads the nominator and denominator stored in the first track.

`Tempo()/ BPM()` - Reads the tempo value and BPM stored in the second track.

`VLQ()` - It extracts 7 bit group from LSB to MSB, stores them in a temporary array, then writes them in reverse with the MSB continuation bit set on all bytes except the last. The first bit is the continuation bit to let us know if there are more bytes to be read. This is used for finding out the timings in the song.

`Control Changes()` - Handles the control changes if any of the sliders on the keyboard were used such as the modulation slider. Unused in our game but still needs to be parsed.

Midi Write

Similar to the Midi parser but writing to a file instead. Many of the same functions are used since we still need to write in sequence.

However since we do not know the actual size of the third track until we go through our recorded notes vector

So we have to manually call when to write which byte (Note On/Note Off, Control Change, Program Change (before and after note placement)) and then add that size amount to the buffer's size.

The VLQ for the note's deltatime for when it plays and stops and the ordering for which one comes after or before was quite difficult due to compression and how they should be ordered by their timestamps. Originally I made a test vector of notes and while this worked for the test, the actual recorded notes didn't due to me not realising that the note's sequence is dependent on their event times and not the actual sequence of notes that were recorded in the vector. This is because you can hold a note and play other notes during the same duration before Note Off is ever called.

Midi Input

Uses a callback function to run on a separate thread to not cause any lag or latency issues with the gameplay. The data it receives from a key press is then passed back to us for detecting the key's note value and velocity (velocity can be 0 and still seen as Note On if the keyboard's volume is set to zero, so it still counts as a key press).

Gameplay

Notes are spawned based on their start time in the note struct that is saved when parsed and the travel distance it will take to reach the Keyboard's sound trigger. We use the note's start time and note's end time to calculate the note's height (duration). When the song begins the notes then move down on the y-axis at their speed determined by the song's BPM.

Ghost notes have a similar setup as to the Falling notes but are overlayed above them and have no triggers assigned to them, so they are only visual.

Database

We pull and push to the server's database in the Result's screen and in the Leaderboard screen.

References

Official Midi Association:

<https://midi.org/>

Official Documentation:

<https://midi.org/spec-detail>

Windows

Windows Multimedia Library (Midi Input Handling, MCI Midi playback)

<https://learn.microsoft.com/en-us/windows/win32/multimedia/midi-functions>

<https://learn.microsoft.com/en-us/windows/win32/multimedia/musical-instrument-digital-interface--midi>

<https://learn.microsoft.com/en-us/windows-hardware/drivers/audio/audio-device-messages-for-midi>

Media Control Interface (Midi playback)

<https://learn.microsoft.com/en-us/windows/win32/multimedia/media-control-interface--mci>

<https://learn.microsoft.com/en-us/windows/win32/multimedia/using-the-mci-midi-sequencer>

<https://learn.microsoft.com/en-us/windows/win32/multimedia/sending-a-command>

Books:

A Beginner's Guide to Midi – R.A. Penfold
(1993 Bernard Babani LTD)

Applications:

Synthesia:

<https://synthesiagame.com/>

Openthesia (free open-source version of Synthesia)

<https://openthesia.pages.dev/>

Repositories:

RTMidi - most popular and lightweight Midi library:

<https://github.com/thestk/rtmidi>

Mido – python Midi library:

<https://mido.readthedocs.io/en/stable/>

Midi Programming – A complete study by Stephane Richard

(Parser Explanation Articles)

<http://www.petesqbsite.com/sections/express/issue18/midifilespart1.html>

<http://www.petesqbsite.com/sections/express/issue19/midifilespart2.html>

<http://www.petesqbsite.com/sections/express/issue20/midifilespart3.html>

Videos:

Javidx9 – Midi Programming Video:

<https://youtu.be/040BKtnDdg0?si=UzKp9w7zF8tkLn6N>

Simon Hutchinson – Midi video series:

<https://www.youtube.com/@SimonHutchinson>